

Augur: Thwarting Traffic Correlation Attacks with Predictive, Real-time Perturbations

IEEE Publication Technology, *Staff*, *IEEE*,

Abstract—Traffic correlation attacks pose a critical threat to low-latency anonymity networks like Tor, enabling adversaries to de-anonymize users by matching statistical patterns in encrypted traffic. However, existing defenses either impose prohibitive performance overheads or, like the current state-of-the-art (SOTA) systems, rely on context-agnostic perturbations that are fundamentally ineffective against sophisticated attacks. We present Augur, a new defense framework that shifts the paradigm from blind, context-agnostic shaping to proactive, predictive perturbation. At its core, Augur utilizes a Transformer-based forecaster to anticipate near-term traffic patterns, allowing it to generate tailored, context-aware perturbations in real time. Our end-to-end training pipeline incorporates a novel differentiable windowing mechanism, making it possible for the first time to directly optimize against complex, window-based attacks by overcoming their non-differentiable operations. We implement and evaluate a full prototype of Augur as a lightweight, client-side Tor Pluggable Transport, demonstrating its practicality with an asynchronous design that ensures low overhead. Extensive experiments show that Augur decisively outperforms prior work. Against the SOTA attack, Augur reduces the F1-score from 0.9756 to just 0.0114 while incurring only 0.07% bandwidth overhead, effectively neutralizing the attack. Our work demonstrates that predictive, context-aware perturbation is a practical and superior defense strategy against advanced traffic correlation attacks.

Index Terms—Traffic correlation attacks, anonymity networks, predictive perturbation.

I. INTRODUCTION

ONLINE anonymity is a cornerstone of secure communication. Tor, the largest low-latency anonymity system, provides critical privacy protection to millions globally [1], [2]. Tor’s low-latency design, however, possesses a fundamental vulnerability. Despite multi-layer encryption, it fails to erase inherent statistical patterns in traffic metadata [3]. This weakness renders Tor susceptible to traffic correlation attacks [4]–[7], a direct threat to its core anonymity guarantee. These attacks require an adversary, such as an ISP or state-level agency, to observe traffic at both entry and exit points. This adversary model is realistic for many users. By matching preserved metadata patterns (e.g., packet timings, sizes, and directions), the adversary links ingress and egress flows, deanonymizing the user.

The evolution of traffic correlation attacks has culminated in architectures that exploit long-range temporal dependencies, rendering traditional defensive strategies obsolete. While early deep learning models pioneered feature extraction, current SOTA attacks (e.g., DeepCoFFEA [8]), leverage robust

window-based aggregation to effectively filter out the static, context-agnostic perturbations typical of deployable real-time defenses (e.g., BLANKET) [9]. This creates a fundamental dichotomy in defense design: robust protection mandates context-aware perturbation to disrupt these complex temporal patterns, yet the “buffer-and-perturb” approach required to compute such optimal noise violates the strict latency constraints of low-latency anonymity networks [10]. Although adversarial training could theoretically automate the learning of these complex patterns, the discrete, non-differentiable voting logic inherent in SOTA attacks severs the gradient flow, impeding direct optimization. Consequently, a critical and unaddressed challenge remains: designing a defense mechanism that can proactively generate sophisticated, context-aware perturbations in real-time, thereby bridging the gap between deployability and resilience against advanced, context-driven adversaries.

Existing countermeasures fail to reconcile these conflicting requirements, leaving the defensive landscape bifurcated into two insufficient paradigms. The first paradigm consists of buffer-and-perturb strategies. These systems operate by buffering an entire traffic trace to compute an optimal perturbation offline. This category includes heavyweight defenses (e.g., Tamaraw [10] and CS-BuFLO [11]) that impose impractical and unappealing overheads [11], [12]. This paradigm also includes powerful, optimization-based adversarial attack methods like Carlini & Wagner (C&W) [13] and Projected Gradient Descent (PGD) [14], which serve as theoretical robustness benchmarks but are non-real-time and thus non-deployable as live defenses. The second paradigm prioritizes deployability through real-time context-oblivious strategies. These systems, such as the SOTA deployable defense BLANKET [9], attempt to balance security and overhead by applying “blind”, pre-computed perturbations on the fly [9]. Their core limitation is the context-agnostic strategy. The generated perturbations are static and one-size-fits-all, and therefore can be easily predicted. Adversaries can simply train a DNN to recognize and filter this statistically uniform noise, since it is detached from the genuine traffic patterns. This leaves a critical, unaddressed gap: a defense must be both deployable (real-time) and context-aware.

To address this gap, we present Augur, a new defense framework shifting the paradigm from blind, context-agnostic obfuscation to proactive, predictive perturbation. Augur employs a Transformer-based forecaster [15] to anticipate near-term traffic patterns. This predictive capability allows it to generate tailored, context-aware perturbations in real time

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

before packets are sent. Unlike prior work applying isolated, packet-by-packet perturbations, Augur computes an optimal perturbation vector for an entire future sequence at once. This approach to predictive defense is our fundamental differentiator. Particularly, we introduce a novel differentiable windowing mechanism to address the non-differentiable issue of SOTA window-based attacks. This surrogate function provides a gradient path through the attack’s non-smooth decision logic, enabling a defense to be trained end-to-end directly against the attack’s internal structure for the first time. We implement and evaluate a full prototype of Augur as a lightweight, client-side Tor Pluggable Transport. Its asynchronous, producer-consumer architecture validates its practical feasibility, ensuring low latency and manageable overhead on commodity hardware. Our contributions are as follows:

- 1) *Predictive Perturbation*. We introduce Augur, the first defense framework that leverages time-series forecasting to generate real-time, context-aware perturbations. The resulting perturbations are tightly coupled with live traffic dynamics, proving more robust than prior “blind” approaches.
- 2) *Differentiable Windowing for End-to-End Defense*. We design and implement a novel differentiable windowing mechanism. This technique resolves a fundamental optimization barrier, enabling a defense to be trained end-to-end directly against the non-differentiable, window-based logic of SOTA attacks.
- 3) *Practical and Deployable System Prototype*. We implement a full prototype of Augur as a high-performance, asynchronous Tor Pluggable Transport, validating the real-world feasibility and deployability of our predictive defense architecture.
- 4) *Comprehensive Evaluations against SOTA*. We conduct extensive experiments showing Augur decisively outperforms prior work. Against the SOTA DeepCoFFEA attack, Augur neutralizes the threat, reducing the F1-score from 0.9756 to 0.0114 while incurring only 0.07% bandwidth overhead.

II. RELATED WORK AND MOTIVATION

A. Related Work

The security of low-latency anonymity networks is characterized by a persistent “arms race” between offensive sophistication and defensive countermeasures.

1) *Anonymity Systems and Correlation Threats*: Low-latency anonymity systems like Tor inherently preserve traffic metadata [1], creating a fundamental vulnerability to traffic correlation attacks [3], [17]. While Foundational research initially quantified this risk, modern attacks have escalated

in sophistication to exploit diverse vectors. Adversaries now successfully target onion service sessions [4], cross-layer timings [18], and cellular metadata [19]. Furthermore, the emergence of automated side-channel discovery tools frequently facilitates these advanced operations [20], [21]. High-latency defenses based on randomization offer resistance but incur prohibitive costs [5]. Consequently, static defenses are consistently outpaced by adversaries that exploit classifier leaks [22] or bandwidth inflation [6], [23].

2) *Attack side*: The threat of traffic correlation evolved dramatically with the integration of DL. Early statistical methods, exemplified by RAPTOR [24], were rendered obsolete, proving ineffective with low accuracy and requiring impractically long observation windows. The paradigm shifted with DeepCorr [25], which first successfully applied CNNs to automate feature extraction and learn correlation patterns directly from noisy traffic traces. Current SOTA attacks, notably DeepCoFFEA [8], elevated this threat by employing metric learning and a resilient, window-based voting mechanism (“amplification”). This design ensures high robustness against the simple, uncorrelated noise that might fool a naive classifier. This offensive vector continues to be refined: ESPRESSO enhances windowing with sensitivity-weighting [26] and Early-MFC targets correlation using only initial packet data [27]. Concurrently, the attack surface has broadened beyond direct flow matching, with specialized attacks targeting Onion Services via non-DL methods like Sliding Subset Sum [4] and others demonstrating that Pluggable Transports are vulnerable to fingerprinting via cross-layer RTTs [18].

3) *Defense side*: Strategies have bifurcated into two main paradigms, as summarized in Table I. Heavyweight, buffer-and-perturb systems (e.g., Tamaraw [10], CS-BuFLO [11], [28]) provide strong theoretical protection by reshaping traffic to fixed distributions, but their impractical latency and bandwidth overheads confine them to theoretical benchmarks [12]. This category also includes optimization-based adversarial attacks like C&W [13] and PGD [14], which are non-real-time but serve as crucial robustness metrics. This impracticality led to the development of deployable, lightweight defenses. The SOTA in this category, BLANKET [9], applies “blind” adversarial perturbations in real-time. However, BLANKET’s strategy is fundamentally context-oblivious. It applies a static, pre-computed perturbation. This approach is ill-suited for adversaries, who can simply train their DNNs to filter this predictable, statistically-uniform noise [29], [30]. Recognizing this critical limitation, the most recent research has explored new defensive avenues, including adaptive (but non-predictive) padding like RUDOLF [31] and KimeraPAD [32], deter-

TABLE I
SUMMARY OF DEFENSE STRATEGIES AND THEIR CHARACTERISTICS.

Defense System	Paradigm	Overhead (Bandwidth)	Overhead (Latency)	Context-Aware
Tamaraw [10]	Buffer-and-Perturb	× (199%–687%)	× (>3.2x)	×
CS-BuFLO [11]	Buffer-and-Perturb	× (130%–180%)	× (2.7x–3.4x)	×
C&W / PGD [13], [14]	Buffer-and-Perturb	Customizable	× Customizable	✓
RegulaTor [16]	Real-Time	✓ (Low: <20%)	✓ (1.05x)	×
BLANKET [9]	Real-Time	✓ (Low: ~10%)	✓ (~50ms)	×
Augur (Ours)	Real-Time	✓ (Very Low: <1%)	✓ (< 1.15x)	✓

ministic protocol-mimicry like DeTorrent [33], and unified implementation frameworks like Maybenot [34].

B. Motivation

The related work reveals a critical stalemate rooted in two flawed defensive paradigms, which are visually contrasted in Fig. 1. The first, buffer-and-perturb (Fig. 1a), consists of non-real-time systems like Tamaraw [10] and CS-BuFLO [11] that impose impractical and unappealing overheads [12]. The second, real-time (Fig. 1a), is deployable but fails because it is “Context-Oblivious” (Fig. 1b).

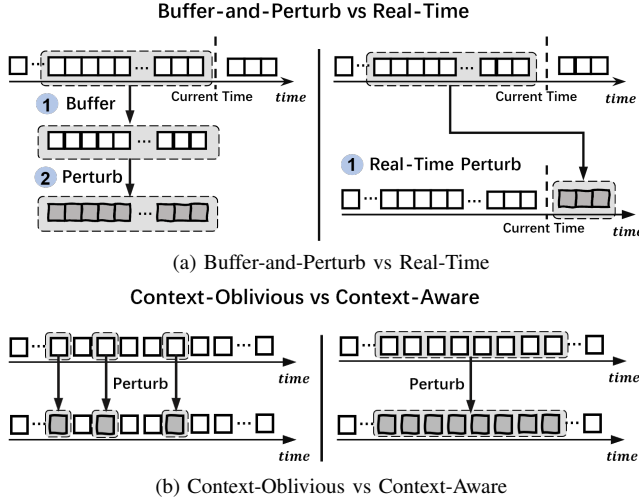


Fig. 1. Defensive Strategy Comparison.

This leads to the core technical conflict: SOTA attacks (e.g., DeepCoFFEA, Sliding Subset Sum) are context-aware, utilizing session-level information and resilient windowing [4], [8], [35]. In sharp contrast, SOTA defenses (e.g., BLANKET) are context-oblivious [9]. This fundamental asymmetry means that modern attacks are specifically designed to defeat the simple noise models used by current deployable defenses, rendering them “useless” in practice [4], [35].

This defense landscape reveals a critical and unaddressed research gap. The field requires a defense that is simultaneously real-time and context-aware. Augur is the first framework designed to meet both criteria. We shift the paradigm from blind, context-agnostic shaping to proactive, predictive perturbation [9]. Augur leverages a Transformer-based forecaster [15] to anticipate traffic dynamics, generating perturbations tightly coupled with the live traffic’s context. This makes it inherently Context-Aware.

III. THREAT MODEL AND DEFENSIVE OBJECTIVES

A. Threat Model

We consider a realistic local passive adversary positioned between the client and the Tor entry node, aiming to compromise unlinkability by correlating ingress and egress traffic. Let $\mathcal{F}_{\text{in}}$ and $\mathcal{F}_{\text{out}}$ denote the sets of observed ingress and egress flows, respectively. To align with the multidimensional nature of deep learning inputs (as utilized in our generator architecture), we formalize each flow $f \in \mathcal{F}$ as a multivariate time-series matrix $\mathbf{X} \in \mathbb{R}^{C \times L}$, where C represents the distinct

traffic variates—specifically inter-packet delays and packet sizes—and L denotes the sequence length. The adversary employs advanced deep learning classifiers to learn a non-linear correlation function $\mathcal{D} : \mathcal{F}_{\text{in}} \times \mathcal{F}_{\text{out}} \rightarrow [0, 1]$, extracting high-dimensional features from these variates to quantify flow association. Consequently, for a specific target ingress flow $\mathbf{X}_{\text{in}}$, the attack succeeds if it correctly identifies the true egress counterpart $\mathbf{X}_{\text{out}}^*$ by maximizing this correlation score: $\mathbf{X}_{\text{out}}^* = \arg \max_{\mathbf{X} \in \mathcal{F}_{\text{out}}} \mathcal{D}(\mathbf{X}_{\text{in}}, \mathbf{X})$.

Consistent with Kerckhoffs’s principle, we assume the adversary possesses full knowledge of the defense mechanism and its parameters but lacks access to the specific stochastic seeds used for perturbation generation. We exclude global adversaries and active manipulation strategies, such as packet dropping or injection, from this scope.

To mitigate this threat effectively, our defense operates under strict deployability and real-time constraints suitable for low-latency anonymity networks. The defender resides on the client side and manipulates traffic solely via packet padding and delaying. We explicitly exclude the dropping or injecting of packets at the network layer to preserve protocol semantics and reliability. Furthermore, the defense must operate in real-time, generating perturbations for current packets without buffering future traffic. This necessitates a predictive approach capable of anticipating traffic dynamics based on historical subsequences. Formally, the defensive objective is to generate a perturbation matrix $\Delta \in \mathbb{R}^{C \times L}$ to produce a perturbed flow $\mathbf{X}'_{\text{in}} = \mathbf{X}_{\text{in}} + \Delta$, such that the adversary’s correlation function $\mathcal{D}(\mathbf{X}'_{\text{in}}, \mathbf{X}_{\text{out}}^*)$ is minimized. This effectively renders the flows statistically indistinguishable and reduces the state-of-the-art attack performance to levels comparable to random guessing.

B. Defensive Objectives

Based on the formalized threat model and the strict operational constraints of low-latency networks, we derive the following five core design objectives.

- 1) *Effectiveness* The primary objective is to minimize the adversary’s correlation advantage. Let G_{θ} denote the perturbation generator and f be the original flow. The system seeks to generate a perturbed flow $f' = f + G_{\theta}(f)$ such that the adversary’s correlation score $\mathcal{D}(f'_{\text{in}}, f^*_{\text{out}})$ approaches the baseline of random guessing. This effectively neutralizes the utility of extracted high-dimensional features.
- 2) *Real-Time Operation*. The perturbation generation must satisfy strict causality constraints. The perturbation δ_t at time t must depend solely on historical observations $H_t = \{(t_i, s_i) \mid t_i < t\}$. We leverage the PatchTST model [36] to forecast future traffic dynamics based on H_t , decoupling inference latency from packet transmission to ensure $\Delta t_{\text{proc}} \approx 0$.
- 3) *Low Overhead*. The framework must optimize the trade-off between security and resource consumption. We define a cost function $\mathcal{L}_{\text{cost}}(\delta)$ based on the L_2 norm of size and timing perturbations. The objective is to minimize $\mathcal{L}_{\text{cost}}$ subject to maintaining the defense efficacy, prioritizing a “time-heavy, size-light” strategy to conserve bandwidth.
- 4) *Transferability*. The learned perturbation strategy must generalize across diverse adversarial architectures. A generator

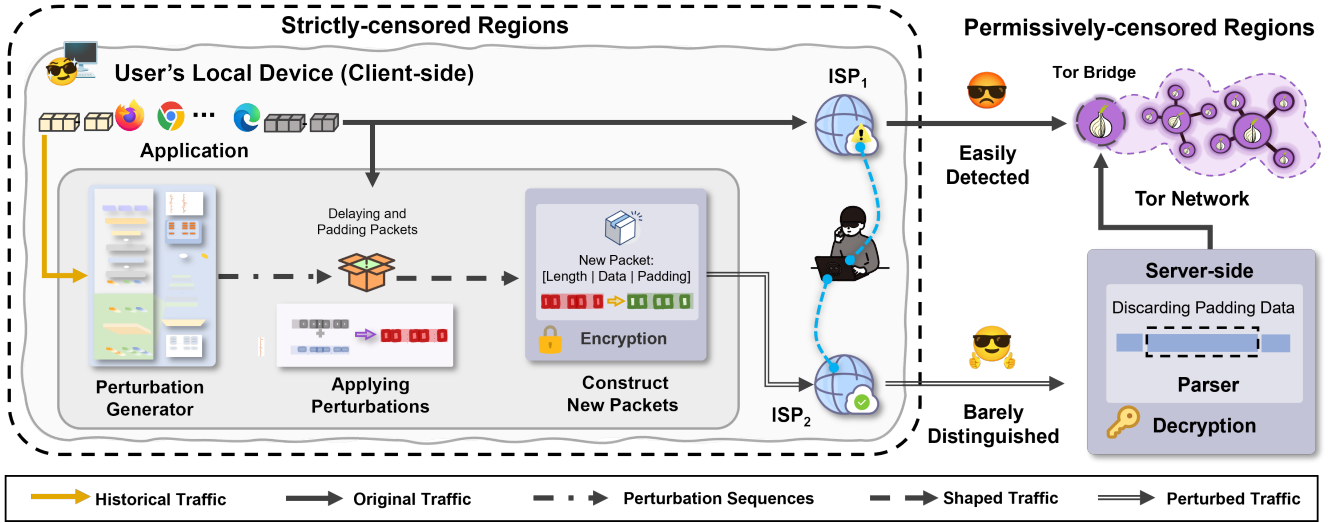


Fig. 2. Overview of Augur.

trained against a specific surrogate model f_A should remain effective against an unknown target model f_B , ensuring robustness against black-box adversaries.

- 5) *Deployability*. The system must integrate seamlessly into the existing Tor ecosystem without requiring server-side perturbations. Following the obfs4 specification [37], Augur operates as a client-side pluggable transport using in-band signaling for metadata transmission. This asymmetric design ensures compatibility with standard Tor relays.

C. Overall Architecture of Augur

Augur is composed of two primary components: a predictive perturbation generator that formulates optimal defensive strategies, and a Pluggable Transport framework that executes them in real time.

D. Core Components

Client-side Perturbation Engine. The engine orchestrates client-side defense using metadata features modeled as a multivariate time-series $\mathbf{X} \in \mathbb{R}^{C \times L}$ (C variates, L sequence length). This metadata-centric approach eliminates deep packet buffering, ensuring lightweight processing via three integrated modules:

- 1) *Generator*: Utilizing a pre-trained PatchTST model G_θ [36], this module monitors historical context $\mathbf{X}_{\text{hist}} \subset \mathbf{X}$ to forecast an optimal perturbation matrix $\Delta \in \mathbb{R}^{C \times L_{\text{out}}}$. Δ consists of pairs $\{(\delta_{t_i}, \delta_{s_i})\}_{i=1}^{L_{\text{out}}}$ explicitly mapping feature channels to delay and padding vectors. To decouple inference latency from network I/O, generation executes asynchronously in a background thread, populating a thread-safe perturbation queue.
- 2) *Applicator*: This module dequeues vector $\delta_i = (\delta_{t_i}, \delta_{s_i})$ to shape outgoing packet P_i . Enforcing a modification-only paradigm (avoiding dummy injection), it appends δ_{s_i} padding bytes and schedules transmission with delay δ_{t_i} via an asynchronous timer. This ensures precise temporal shaping without stalling the event loop.
- 3) *In-band Signaling Encapsulator*: To facilitate transparent decoding, we implement an obfs4-inspired framing

protocol [37]. The module constructs a composite frame $P'_i = L_{\text{real}} \parallel P_{\text{data}} \parallel P_{\text{pad}}$ (header, payload, padding) and encrypts it using session key K_{ses} . The resulting $\mathcal{E}_{K_{\text{ses}}}(P'_i)$ presents a statistically opaque block to network observers.

Server-side Decapsulation Frontend. The server architecture prioritizes stateless operation and throughput via two lightweight modules:

- 1) *Decryptor*: Recovers the plaintext frame P'_i from the encrypted stream using the symmetric session key.
- 2) *Parser*: Reads the L_{real} header to identify payload boundaries, extracting P_{data} for forwarding while discarding padding P_{pad} . This asymmetric design offloads perturbation complexity to the client, significantly lowering deployment barriers.

IV. PERTURBATION GENERATION

The defensive capability of Augur hinges on generating perturbations that are structurally disruptive yet context-aware. This requirement is met by a generator model designed to learn optimal evasion strategies through end-to-end optimization. The following subsections elaborate on the core components of this generator: its predictive architecture, the guiding objective functions, and the differentiable mechanisms that ensure consistent gradient flow.

A. Architecture of Perturbation Generator

Our predictive architecture consists of four tightly integrated modules: input representation and normalization, variate-independent patch embedding, a progressive encoder composed of alternating Transformer blocks and upsampling layers, and a forecasting head. This hierarchical design enables the model to efficiently capture both local and global temporal dependencies, while accommodating the heterogeneous and multi-scale nature of real-world network traffic. Fig. 3 illustrates the data flow and main tensor transformations in the model.

Patching and Embedding. Due to the fundamentally different temporal dynamics of packet size and IPD, we adopt a variate-independent patch embedding strategy. The normalized

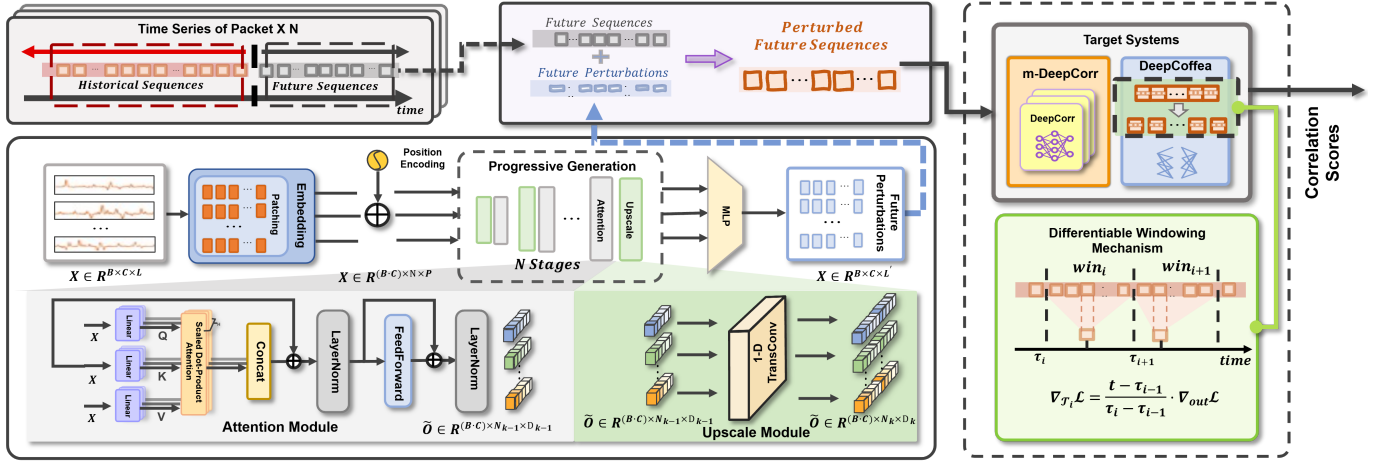


Fig. 3. Architecture of Augur's perturbation generator.

input is partitioned along the temporal axis into N overlapping patches, each of length P and stride S , where $N = \lfloor \frac{L-P}{S} \rfloor + 1$. This yields a 4D patch tensor $X_{\text{patch}} \in \mathbb{R}^{B \times C \times N \times P}$. For each channel c , an independent linear projection $W_p^{(c)} \in \mathbb{R}^{P \times D}$ with bias $b_p^{(c)} \in \mathbb{R}^D$ is applied:

$$E_{b,n,c,:} = X_{b,c,n,:}^{(\text{patch})} W_p^{(c)} + b_p^{(c)}. \quad (1)$$

A learnable positional encoding $W_{\text{pos}} \in \mathbb{R}^{N \times D}$ is then added. We denote the tensor reshape operation as $\mathcal{R}(\cdot)$. The entire patch embedding process is denoted by $\mathcal{P}(\cdot)$, which includes patching, projection (Eq. (1)), reshaping, and adding positional encoding. The embedded patches z_0 are:

$$z_0 = \mathcal{P}(X) = \mathcal{R}(E, (B \cdot C, N, D)) + W_{\text{pos}}, \quad (2)$$

where E is the tensor of projected patches from Eq. (1), and $z_0 \in \mathbb{R}^{(B \cdot C) \times N \times D}$. This design allows each feature to learn a dedicated temporal representation and compresses the effective sequence length, easing subsequent attention computation.

Variate-independent Progressive Encoder. To enforce channel independence, our encoder first treats each of the C traffic variates as an independent instance within the batch dimension. This ensures self-attention is computed separately for each variate. The encoder then progressively refines temporal resolution by alternating between Transformer blocks and upsampling layers. We denote the upsampling layer as $\mathcal{U}(\cdot)$. Given input z_0 , the forward pass for each stage $k = 0, \dots, K-1$ is recursively defined as: $h_k = \text{Block}_k(z_k)$, $z_{k+1} = \mathcal{U}_k(h_k)$. We denote the full cascade of K encoder stages as $\mathcal{K}(\cdot)$, such that $z_K = \mathcal{K}(z_0)$. Each Block_k is a standard transformer encoder block [15], using layer normalization, denoted $\mathcal{LN}(\cdot)$:

$$\begin{aligned} \tilde{h}_k &= \mathcal{LN}(z_k + \text{MSA}(z_k)), \\ h_k &= \mathcal{LN}(\tilde{h}_k + \text{FFN}(\tilde{h}_k)), \end{aligned} \quad (3)$$

where multi-head self-attention (MSA) is defined as $\text{MSA}(h_k) = \mathcal{C}(\text{head}_1, \dots, \text{head}_H) W^O$, with $\mathcal{C}(\cdot)$ denoting concatenation and $\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i$. Q_i , K_i , and V_i are projections of h_k ($Q_i = h_k W_i^Q$, $K_i = h_k W_i^K$, $V_i = h_k W_i^V$).

Each $\mathcal{U}_k$ is a 1D transposed convolution (or interpolation) that doubles the sequence length ($N_k \rightarrow 2N_k$) and halves

the channel dimension ($D_k \rightarrow D_{k+1} = D_k/2$): $z_{k+1} \in \mathbb{R}^{(B \cdot C) \times 2N_k \times (D_k/2)}$. After K stages, we obtain the final high-resolution representation $z_K \in \mathbb{R}^{(B \cdot C) \times N_{\text{out}} \times D_{\text{out}}}$, where N_{out} matches the target temporal resolution.

Forecasting Head. The forecasting head maps the final encoder representation z_K to the final perturbation sequence $\hat{Y}$. For each of the C variates, the head flattens the sequence and feature dimensions, denoted $\mathcal{F}(\cdot)$, and then applies an independent linear layer, $\mathcal{L}(\cdot)$, to project the resulting vector to the target prediction length L' . This process yields the final output $\hat{Y} \in \mathbb{R}^{B \times C \times L'}$.

The entire inference pipeline, combining the patch embedding $\mathcal{P}(\cdot)$ (Eq. (2)), the progressive encoder cascade $\mathcal{K}(\cdot)$, and the forecasting head ($\mathcal{L}, \mathcal{F}$), is thus:

$$\hat{Y} = \mathcal{L}(\mathcal{F}(\mathcal{K}(\mathcal{P}(X)))). \quad (4)$$

By combining feature-specific embedding, hierarchical progressive encoding, and precise upsampling, this architecture achieves both computational efficiency and the ability to generate physically consistent network flow predictions.

B. Differentiable Windowing Mechanism

A critical component of SOTA attacks (e.g., DeepCoFFEA [8] and Sliding Subset Sum [4]) is its temporal windowing mechanism, which segments traffic traces into discrete windows based on cumulative packet inter-arrival times. In standard implementations, this process relies on non-differentiable operations—specifically searchsorted for index identification and discrete array slicing for feature extraction. These operations block the gradient flow from the attack's decision logic back to the perturbation generator, precluding end-to-end optimization. To address this, we implement a fully differentiable windowing pipeline comprising three novel components: surrogate boundary search, interpolation-based sampling, and temporal soft masking.

1) Differentiable Boundary Search: The standard window segmentation function, denoted as $\mathcal{S}(\cdot)$, maps a continuous timestamp t to a discrete index i within the sorted cumulative timestamp sequence $\mathcal{T}$. This mapping is a step function with zero gradients almost everywhere. We introduce a custom

autograd function, $\mathcal{S}_{\text{diff}}(\cdot)$, to approximate this behavior differentially. In the forward pass, $\mathcal{S}_{\text{diff}}(\cdot)$ behaves identically to the standard `searchsorted` operator, returning the precise index i such that $\bar{T}_{i-1} \leq t < \bar{T}_i$. In the backward pass, we approximate the gradient using linear interpolation. Specifically, we distribute the gradient of the downstream loss $\mathcal{L}$ to the timestamps $\bar{T}_{i-1}$ and $\bar{T}_i$ enclosing the target boundary t . The gradient weights are inversely proportional to the temporal distance between the timestamp and the boundary: $\alpha = \frac{t - \bar{T}_{i-1}}{\bar{T}_i - \bar{T}_{i-1}}$, $\nabla_{\bar{T}_i} \mathcal{L} = \alpha \cdot \nabla_{\text{out}} \mathcal{L}$, $\nabla_{\bar{T}_{i-1}} \mathcal{L} = (1 - \alpha) \cdot \nabla_{\text{out}} \mathcal{L}$. This surrogate gradient provides the generator with directional feedback, indicating how to adjust IPDs to push packets across window boundaries.

2) *Continuous Window Extraction*: Identifying the boundary indices alone is insufficient; the extraction of features within these boundaries must also be differentiable. Standard array slicing is a discrete operation that breaks the computation graph. We propose a continuous sampling approach to overcome this. Instead of slicing, we treat the packet feature sequence X as a continuous signal. Given a floating-point start index I_{start} derived from $\mathcal{S}_{\text{diff}}(\cdot)$, we generate a sampling grid $q_j = I_{\text{start}} + j$ for $j \in \{0, \dots, W_{\text{len}} - 1\}$, where W_{len} is the target window length. We extract the window features $\hat{X}_j$ via linear interpolation at these floating-point coordinates: $\hat{X}_j = (1 - \eta) \cdot X_{\lfloor q_j \rfloor} + \eta \cdot X_{\lceil q_j \rceil}$, where $\eta = q_j - \lfloor q_j \rfloor$. This formulation ensures that small perturbations in the predicted timestamps result in continuous changes in the extracted window features.

3) *Temporal Validity Soft Masking*: To enforce the temporal constraints of the window end boundary I_{end} , we apply a differentiable soft mask. We compute a validity mask M using a sigmoid function with a steepness factor γ (set to 10.0 in our implementation):

$$M_j = \sigma(\gamma \cdot (I_{\text{end}} - q_j)). \quad (5)$$

The final window features are computed as $\tilde{X}_j = \hat{X}_j \cdot M_j$. This soft masking mechanism penalizes features that drift beyond the valid temporal boundary I_{end} , thereby establishing a complete gradient path from the classifier output back to the raw packet timestamps.

C. Adversarial Training Strategy

The perturbation generator G_θ aims to optimize a multi-objective loss $\mathcal{L}_{\text{adv}}$. Let $f_{\mathcal{T}}$ denote the target system. The loss is:

$$\mathcal{L}_{\text{adv}} = \mathcal{L}_{\text{def}}(f_{\mathcal{T}}(X + \delta)) + \mathcal{L}_{\text{cost}}(\delta), \quad (6)$$

where X is the original traffic, $\mathcal{L}_{\text{def}}$ is the adaptive defense loss, and $\mathcal{L}_{\text{cost}}$ constrains the perturbation's magnitude for stealth.

Defense against Binary Classifiers ($f_{\mathcal{A}}$). Target models like DeepCorr [25] are CNN-based binary classifiers, denoted $f_{\mathcal{A}}$, trained via Binary Cross-Entropy (BCE). To defend, we target correlated flow pairs ($y = 1$) and apply a label-flipping strategy ($y_{\text{target}} = 0$). The defense objective minimizes the loss for the *incorrect* label:

$$\mathcal{L}_{\text{def}}^{\mathcal{A}} = -\log(1 - f_{\mathcal{A}}(X + \delta)). \quad (7)$$

Minimizing this loss forces the predicted probability $p' = f_{\mathcal{A}}(X + \delta)$ toward 0, causing a misclassification.

Defense against Metric Learning Models ($f_{\mathcal{B}}$). Attacks like DeepCoFFEA ($f_{\mathcal{B}}$) use a non-differentiable, multi-stage pipeline [8]. They segment flows into N_{win} windows and compute similarity scores $\text{Sim}_i = \text{sim}(G(t_i), H(x_i))$, where $\text{sim}(\cdot)$ denotes cosine similarity. They then aggregate binary votes $v_i = \mathbb{I}(\text{Sim}_i \geq \tau)$, using the indicator function $\mathbb{I}(\cdot)$. The final classification $f_{\mathcal{B}}(t, x) = \mathbb{I}(\sum v_i \geq k_{\text{value}})$ is non-differentiable.

Our defense seeks a perturbation δ such that $f_{\mathcal{B}}(t', x') = 0$. We use a surrogate loss, defining the hinge function as $\mathcal{H}(\cdot) = \max(0, \cdot)$. The loss penalizes window similarities Sim'_i that exceed a margin $m < \tau$:

$$\mathcal{L}_{\text{def}}^{\mathcal{B}} = \frac{1}{N_{\text{win}}} \sum_{i=1}^{N_{\text{win}}} \mathcal{H}(\text{sim}(G(t'_i), H(x'_i)) - m). \quad (8)$$

Minimizing this loss pushes Sim'_i below the margin, reducing the total vote count and forcing a misclassification.

Perturbation Constraints. To ensure perturbations are physically feasible and stealthy, we impose two constraints.

1) *Physical Realizability*. We enforce non-negative constraints. Size perturbations ($\delta_s \geq 0$) only permit packet padding. Timing perturbations ($\delta_t \geq 0$) only permit positive delays. These are enforced via ReLU or `clamp(min = 0)` operations.

2) *Perturbation Cost*. To ensure stealth, we adopt an adaptive cost function $\mathcal{L}_{\text{cost}}$ using a weighted sum of normalized L_2 norm ratios:

$$\mathcal{L}_{\text{cost}}(\delta) = \lambda_s \frac{\|\delta_s\|_2}{\|X_s\|_2} + \lambda_t \frac{\|\delta_t\|_2}{\|X_t\|_2}, \quad (9)$$

where λ_s and λ_t are tunable hyperparameters controlling the effectiveness-stealth trade-off.

The objectives for $f_{\mathcal{A}}$ (positive samples) and $f_{\mathcal{B}}$ are defined by combining Eq. (9) with Eq. (7) and Eq. (8), respectively.

$$\begin{aligned} \mathcal{L}_{\text{adv}}^{\mathcal{A}} &= -\log(1 - f_{\mathcal{A}}(X + \delta)) + \lambda_s \frac{\|\delta_s\|_2}{\|X_s\|_2} + \lambda_t \frac{\|\delta_t\|_2}{\|X_t\|_2}, \\ \mathcal{L}_{\text{adv}}^{\mathcal{B}} &= \frac{1}{N_{\text{win}}} \sum_{i=1}^{N_{\text{win}}} \mathcal{H}(\text{sim}(\cdot) - m) + \lambda_s \frac{\|\delta_s\|_2}{\|X_s\|_2} + \lambda_t \frac{\|\delta_t\|_2}{\|X_t\|_2}. \end{aligned} \quad (10)$$

Training Procedure. The proposed differentiable windowing mechanism ($\mathcal{S}_{\text{diff}}(\cdot)$, Section IV-B) ensures a continuous gradient flow from the adversarial loss $\mathcal{L}_{\text{adv}}$ back to the generator parameters θ_{gen} . This connectivity facilitates rigorous end-to-end adversarial training. In this workflow, raw sessions are segmented into historical (X_{hist}) and future (X_{future}) sequences. The generator $G_{\theta_{\text{gen}}}$ processes X_{hist} to produce a perturbation δ_{pred} , which is then applied to X_{future} . The resulting perturbed flow is passed to the target model ($f_{\mathcal{A}}$ or $f_{\mathcal{B}}$) to compute the defense loss $\mathcal{L}_{\text{def}}$. Finally, the total adversarial loss $\mathcal{L}_{\text{adv}}$ is backpropagated to update θ_{gen} via the Adam optimizer.

V. PROTOTYPE IMPLEMENTATION OF AUGUR

To validate the feasibility of our design and rigorously evaluate its performance, we have developed a functional prototype of Augur. The prototype is implemented in Python

and built atop the Twisted framework. To facilitate community reproduction and further study, our prototype, training scripts, and experimental data have been publicly released on <https://github.com/Ni-Xilin/Augur>

A. Core Architecture

Asynchronous Client-side. Augur client constitutes the core component of the defense system. Its implementation is based on the asynchronous event-driven architecture of `obfsproxy` and deeply integrates our perturbation generator.

Core Framework and Asynchronous I/O. The client runs on Twisted’s asynchronous event loop, a crucial architectural choice enabling non-blocking handling of network events. All network data transmission and reception occur within asynchronous callbacks scheduled by Twisted’s reactor.

Integration and Asynchronous Execution of the Perturbation Generator. Our perturbation generator is computationally intensive. Direct synchronous invocation on the main networking thread causes blocking delays, severely impacting network responsiveness. To overcome this bottleneck, we adopt an asynchronous producer-consumer architecture:

- 1) **Perturbation Queue:** We maintain a thread-safe perturbation queue implemented as `collections.deque`, acting as a buffer between the main network thread (consumer) and a background worker thread (producer).
- 2) **Asynchronous Generation Tasks:** The generator is never invoked synchronously in the main I/O loop. Instead, computationally heavy generation tasks are dispatched to background worker threads through Twisted’s thread pool.
- 3) **Watermark Trigger Mechanism:** After dequeuing a perturbation instruction, the main thread inspects the remaining queue size. If it falls below a preset watermark, an asynchronous generation task is triggered non-blockingly. This ensures that compute-intensive inference never delays network I/O.

Protocol and Encryption Layer. Inspired by `obfs4`’s in-band signaling, we implement a complete protocol stack. We reuse the verified Diffie-Hellman key exchange module from `obfsproxy` to securely negotiate session keys between client and server. During transmission, payloads composed of `[real_length | real_data | padding]` are encrypted using AES-GCM. Both inner payload and outer encrypted frames are prefixed with 2-byte network-byte-order length headers, forming the final transmitted data frames.

Lightweight Server-side Architecture. The Augur server is designed to be lightweight and simple, strictly following a stateless deobfuscation principle. It requires no knowledge of the underlying perturbation logic. At its core lies a streaming state machine parser continually processing the TCP stream buffer. It first reads 2 bytes to obtain the length of the upcoming encrypted data block. Once an encrypted block is received, it is decrypted. The decrypted data is similarly parsed: the first 2 bytes indicate the true payload length, allowing precise extraction of user data while discarding padding bytes. This minimalistic design significantly reduces server-side CPU and memory overhead, simplifying deployment and maintenance.

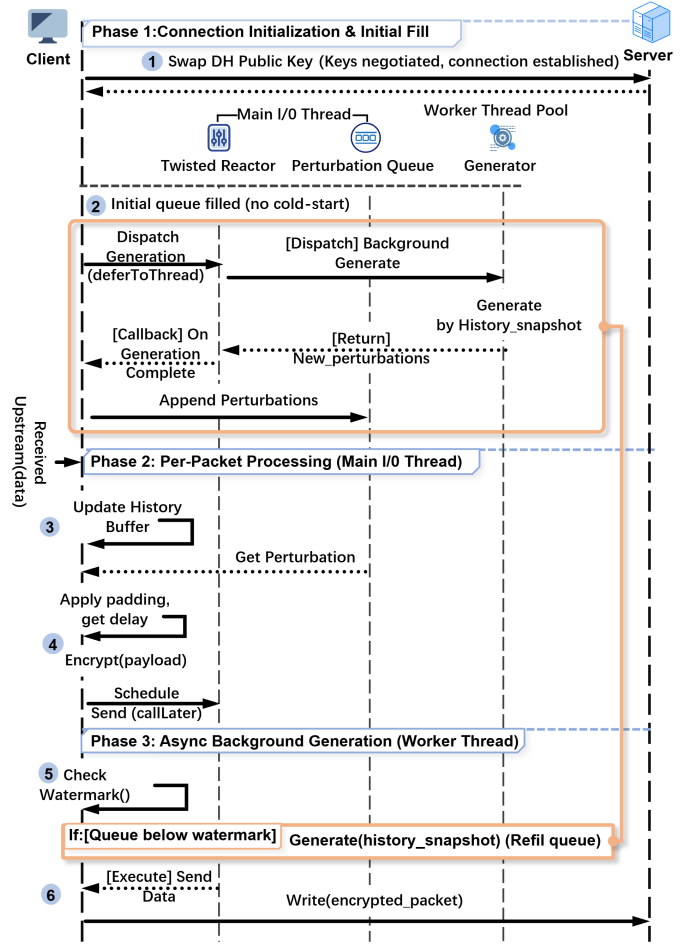


Fig. 4. Real-time Workflow of Augur.

B. Real-Time Workflow

Augur’s runtime architecture is a non-blocking, event-driven system built on Twisted’s asynchronous I/O model. The fundamental goal is to separate compute-heavy model inference from latency-sensitive network operations, enabling complex AI integration without latency or throughput degradation.

The workflow comprises three sequential phases: connection initialization, per-packet processing on the main thread, and asynchronous background perturbation generation. Fig. 4 illustrates this asynchronous workflow.

Phase 1: Connection Initialization and Initial Queue Filling. Upon establishing a new Tor connection, the client performs a Diffie-Hellman handshake to securely negotiate session keys. After handshake completion, the client preemptively triggers an asynchronous perturbation generation, filling the perturbation queue in advance and eliminating cold-start delays.

Phase 2: Per-Packet Processing on Main I/O Thread. All packet processing occurs on Twisted’s main I/O thread to minimize latency. When a packet arrives from the Tor client, a history buffer implemented as a NumPy ring buffer is updated, maintaining short-term traffic context by evicting the oldest features and appending new ones.

Next, the client dequeues a perturbation instruction. If the perturbation queue size falls below a watermark and no generation task is pending, it asynchronously triggers a new

generation process.

The perturbation instruction is applied by padding the packet. The packet is then encapsulated with in-band signaling headers and encrypted under AES-GCM.

Finally, transmission is asynchronously scheduled via `reactor.callLater` according to the delay specified in the perturbation, fully decoupling perturbation timing from network I/O.

Phase 3: Asynchronous Background Perturbation Generation. This runs concurrently in a dedicated worker thread dispatched via `deferToThread`. The worker snapshots current history buffers to avoid concurrency issues and executes computationally heavy inference. Once complete, the new perturbations are passed back using Twisted’s Deferred mechanism and appended to the perturbation queue, ensuring a continuous supply while preserving system responsiveness.

High performance and low latency are maintained through an asynchronous architecture that decouples computationally intensive perturbation generation from network I/O. Furthermore, inspired by `obfs4`, Augur’s asymmetric, client-centric design utilizes in-band signaling, simplifying deployment by requiring only a lightweight, stateless server component. Finally, stealthiness is enhanced by relying exclusively on fine-grained delays and padding for obfuscation, thus preserving fundamental flow characteristics against statistical detection and avoiding the potential fingerprints introduced by dummy packet injection.

VI. EVALUATION

A. Experimental Setup

Datasets. Our evaluation uses two public, real-world Tor traffic benchmarks. The `DeepCorr` dataset contains 25,000 ingress/egress flow pairs (timestamps, sizes, directions) from a crawl of the Alexa top 50,000 sites. The `DeepCoFFEA` dataset, collected between 2021–2022, reflects a more modern network environment and is structured for windowing and metric learning attacks.

Target Systems. We evaluate our method against three SOTA DNN-based benchmarks.

DeepCorr. A flow correlation system that uses a CNN to learn a correlation function directly from traffic features (IPDs and sizes), outperforming conventional methods. We use the `DeepCorr-300` configuration, which analyzes the first 300 packets.

M-DeepCorr. An enhancement to `DeepCorr` that reduces overhead and false positives via a staged filtering architecture. A lightweight model (DC-100) rapidly filters candidates for a more powerful model (DC-700). We adopt the optimal $100 \rightarrow 700$ configuration.

DeepCoFFEA. An advanced system that overcomes $O(N^2)$ scalability issues. It employs (1) metric learning with Feature Embedding Networks (FENs) to map traffic to a low-dimensional space, reducing complexity to $O(2N)$, and (2) an “Amplification” technique using window-based voting to improve detection rates and lower the FPR. We use the optimal hyperparameters from the original work.

B. Experimental Results

To benchmark Augur’s performance, we evaluate it against a diverse suite of SOTA defenses. These baselines span multiple paradigms, from classic white-box adversarial attacks to specialized real-time defenses for traffic analysis. A primary distinction lies in their operational model: traditional methods like C&W and PGD utilize buffer-and-perturb optimization on complete traffic traces. In contrast, BLANKET, the SOTA in real-time defense, applies pre-computed, context-agnostic perturbations on the fly. Augur bridges this gap, generating context-aware perturbations in real-time as traffic flows. We compare Augur against the following established baselines:

- 1) *Random Morphing (RM)*: A non-optimization-based approach that perturbs packet IPD and size features by adding Gaussian noise. This method is derived from Random Mutation [29], which is not a naive or unprincipled attack, but has been recognized in prior work [29], [38].
- 2) *White-Box Attacks (C&W [13], PGD [14], Iterative Fast Gradient Sign Method (IFGSM) [39], Jacobian-based Saliency Map Attack (JSMA) [40], [41])*: Powerful, optimization-based attacks serving as benchmarks for robustness. C&W and PGD are considered gold standards. For JSMA, which is L0-norm oriented, we strengthen it by allowing perturbations on up to 25% of packets. For all attack-based baselines except BLANKET, we constrain perturbations to a relative L2 norm of less than 0.15 for both time ($L2_{\text{time}}$) and size ($L2_{\text{size}}$) to ensure a fair comparison.
- 3) *BLANKET [9]*: The SOTA real-time defense. We configure it with the optimal parameters from the original paper for its best-case performance.

Effectiveness against SOTA Attacks. Fig. 5 plots the ROC curves, demonstrating Augur’s effectiveness. Augur’s defense substantially degrades the performance of all three attack models, pushing their curves toward the random-guessing baseline. This is most evident against `DeepCoFFEA` in Fig. 5c, where Augur’s ROC curve is nearly flat, while all other methods maintain high true positive rates at low FPRs.

Table II quantifies this degradation and groups methods by deployability. The buffer-and-perturb baselines, such as C&W and PGD, serve as theoretical robustness benchmarks; they require offline optimization on complete traces and are not practical for real-time deployment. These non-deployable methods set a high bar, achieving F1-scores as low as 0.0078 by JSMA against `DeepCorr` and 0.0290 by PGD against `m-DeepCorr`.

The critical comparison is within the Real-time Defenses group. Augur decisively outperforms BLANKET, the only other deployable SOTA defense, across all scenarios. Against `m-DeepCorr` at a 10^{-4} FPR, Augur’s F1-score of 0.0557 is 7x lower, or better, than BLANKET’s 0.4000.

This superiority culminates in the virtual nullification of the state-of-the-art `DeepCoFFEA` attack. At a 10^{-4} FPR, Augur’s F1-score plummets to 0.0114, while BLANKET remains highly vulnerable at 0.4234. Notably, Augur’s real-time performance here not only beats its deployable counterpart but also exceeds that of the theoretical buffer-and-perturb

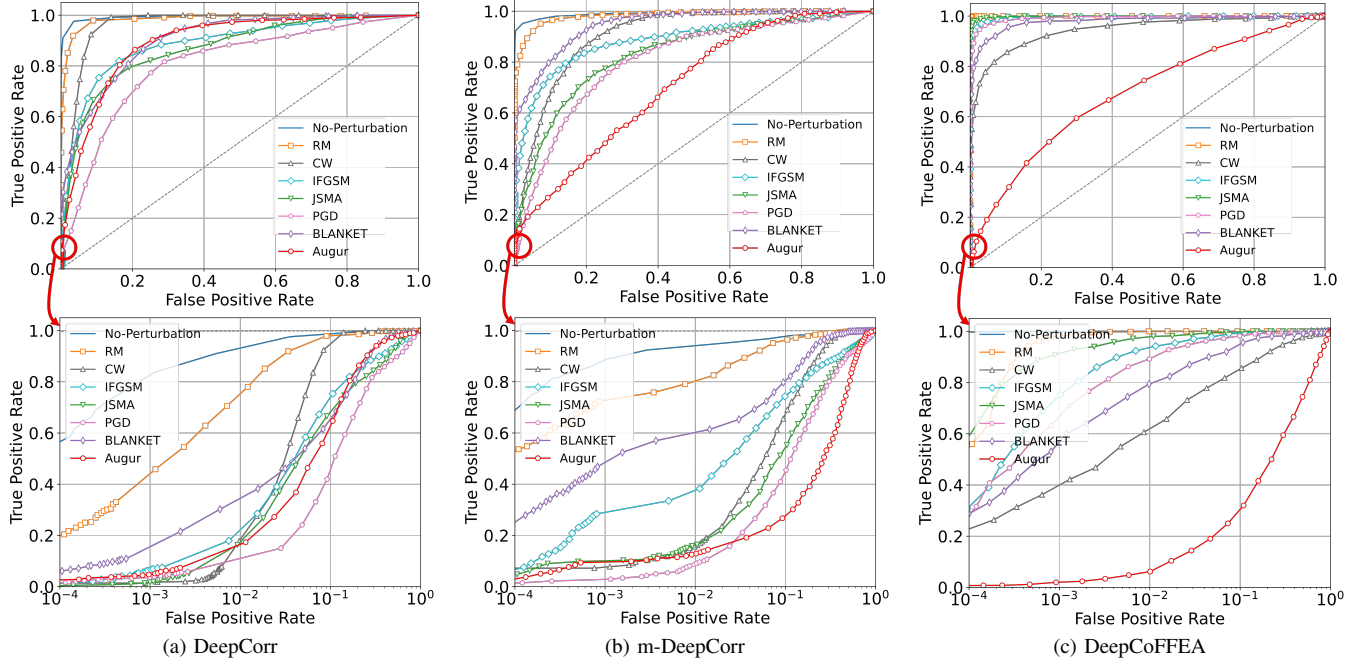


Fig. 5. Performances of Augur and baseline defenses

TABLE II
DETAILED EVALUATION OF AUGUR AGAINST SOTA DEFENSES AT DIFFERENT FALSE POSITIVE RATE (FPR) THRESHOLDS

Category	FPR	Defense Method	Target Attack System											
			DeepCorr						m-DeepCorr					
			PRE	REC	F1	$L2_{time}R$	$L2_{size}R$		PRE	REC	F1	$L2_{time}R$	$L2_{size}R$	
-	10^{-3}	-	0.8064	0.8298	0.8180	-	-		0.8164	0.8850	0.8493	-	-	
Buffer-and-Perturb	10^{-3}	RM	0.6839	0.4310	0.5288	0.1289	0.1325		0.7840	0.7260	0.7539	0.1298	0.1327	
		C&W	0.0706	0.0152	0.0249	0.0969	0.1325		0.2747	0.0757	0.1187	0.1128	0.1161	0.4390
		IFGSM	0.2405	0.0631	0.0999	0.1323	0.0705		0.5939	0.2910	0.3906	0.1329	0.0591	0.5955
		JSMA	0.0744	0.0160	0.0264	0.1416	0.0696		0.3300	0.0980	0.1511	0.1469	0.0618	0.6465
		PGD	0.1568	0.0370	0.0599	0.1290	0.0650		0.1195	0.0270	0.0440	0.1337	0.0271	0.5729
Real-Time	10^{-3}	BLANKET	0.4379	0.1552	0.2291	0.5238	0.3900		0.7116	0.4910	0.5811	0.6055	0.2035	0.5318
		Augur(ours)	0.1984	0.0494	0.1984	0.1343	0.0089		0.3265	0.0968	0.1493	0.1463	0.0367	0.0380
-	10^{-4}	-	0.9644	0.5425	0.6944	-	-		0.9714	0.6790	0.7993	-	-	
Buffer-and-Perturb	10^{-4}	RM	0.9075	0.2044	0.3337	0.1289	0.1325		0.9614	0.5188	0.6739	0.1298	0.1327	0.9068
		C&W	0.3846	0.0126	0.0245	0.0969	0.1325		0.7733	0.0686	0.1261	0.1128	0.1161	0.8188
		IFGSM	0.4118	0.0140	0.0271	0.1323	0.0705		0.5939	0.0700	0.1284	0.1329	0.0591	0.8634
		JSMA	0.1667	0.0040	0.0078	0.1416	0.0696		0.6825	0.0430	0.0809	0.1469	0.0618	0.9228
		PGD	0.4286	0.0150	0.0290	0.1290	0.0650		0.4286	0.0150	0.0290	0.1337	0.0271	0.8418
Real-Time	10^{-4}	BLANKET	0.7468	0.0591	0.1095	0.5238	0.3900		0.9273	0.2550	0.4000	0.6055	0.2035	0.8494
		Augur(ours)	0.5745	0.0272	0.0520	0.1343	0.0089		0.5918	0.0292	0.0557	0.1463	0.0367	0.1071

benchmarks. Augur’s 0.0114 F1-score is over 30x better than the best non-deployable method, C&W, which scored 0.3542. Augur is the only method to effectively counter DeepCoFFEA. Critically, Augur achieves this dual victory with the lowest size perturbation overhead, or $L2_{size}R$, of any method, at only 0.0038 for DeepCoFFEA.

Time-Dominant Perturbation Strategy. Augur’s efficiency is an emergent property of its design. The multi-objective loss function incentivizes the generator to maximize defensive impact while penalizing perturbation cost. As shown in Table II, this leads to an autonomously learned strategy: size overhead, or $L2_{size}R$, is consistently minimal, e.g., 0.0089 vs. DeepCorr, 14x lower than C&W. The generator learns that aggressive

size perturbations yield diminishing returns and converges on a “time-heavy, size-light” strategy.

Efficacy Scaling with Sequence Length Augur’s predictive nature ensures its defense efficacy scales with the attack’s sequence length. Against a short-sequence model like DeepCorr, the 96-packet history required for prediction means only 68% of the flow is effectively perturbed. For a long-sequence adversary like DeepCoFFEA, this 150-packet initialization is a negligible fraction of the total trace, allowing nearly the entire flow to be perturbed. This increasing effective perturbation ratio against longer sequences explains its outsized performance against modern adversaries.

Eradicating Distinguishable Patterns. We qualitatively

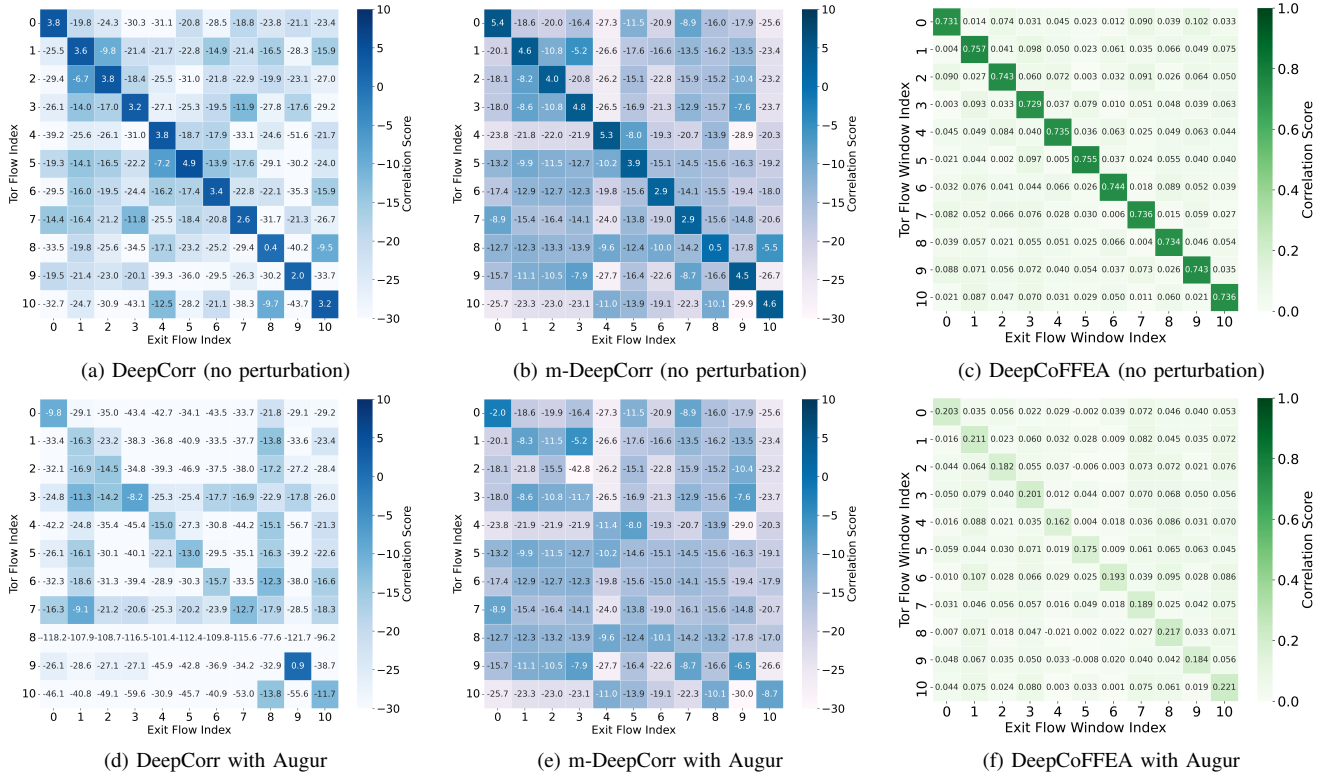


Fig. 6. Impact of Augur on correlation score matrices.

assess Augur’s capability by visualizing the attack models’ raw confidence scores in correlation score matrices, shown in Fig. 6. For DeepCorr and m-DeepCorr, scores are pre-sigmoid pairwise comparisons; for DeepCoFFEA, scores are the average correlation across all windows.

Without Augur. In the undefended scenario detailed in Figures 6a, 6b, and 6c, all attack models produce a clear, exploitable pattern: scores on the main diagonal are significantly higher than off-diagonal scores. This diagonal dominance provides a distinct signal for identifying correlated pairs.

With Augur. With Augur applied, Figures 6d, 6e, and 6f show this pattern is eliminated. The key effect is the suppression of the high-score signal on the main diagonal; scores of correctly matched pairs are reduced to the same level as uncorrelated pairs, rendering them indistinguishable.

An artifact appears in the m-DeepCorr matrix, Fig. 6e: off-diagonal, or uncorrelated, scores remain largely unchanged. This stems from m-DeepCorr’s multi-stage architecture; its first-stage filter operates on only the initial 100 packets, dismissing most uncorrelated pairs before Augur’s main perturbations, derived from a 96-packet history, take full effect. The final scores for these pairs are thus determined by the unaffected initial screening.

Despite this artifact, Augur successfully flattens the score distribution, neutralizing the reliability of any decision threshold and fundamentally undermining the attack. This qualitative suppression of the diagonal signal directly explains the quantitative collapse of the F1-scores reported in Table II.

Efficiency Evaluation: Training Efficiency and Real-Time Overhead. To comprehensively evaluate the practical performance impact of Augur, this section quantifies its efficiency in a realistic client-side environment. These metrics

TABLE III
TRAINING EFFICIENCY AND REAL-TIME OVERHEAD OF AUGUR:
SUMMARIZING ONE-TIME OFFLINE TRAINING COSTS AND REAL-TIME
CLIENT-SIDE COMPUTATIONAL OVERHEAD

Aspect	Metric	Value
<i>Offline Training (One-time cost)</i>		
Generator Training	vs. DeepCorr-300 (GPU)	< 10 min
	vs. m-DeepCorr (GPU)	< 15 min
	vs. DeepCoFFEA (GPU)	< 30 min
<i>Real-time Client Overhead (Per-instance)</i>		
Async Generation	95th Pctl Latency (DeepCorr, CPU)	21.28 ms
	95th Pctl Latency (m-DeepCorr, CPU)	19.29 ms
	95th Pctl Latency (DeepCoFFEA, CPU)	13.65 ms
Resource Footprint	Avg CPU Usage (8-core CPU)	~700%
	Peak RAM Usage	510 - 554 MB

collectively validate that Augur meets its core design goals of “Low Overhead” and “Real-time Operation”.

Offline Training Time. The Augur perturbation generator is trained in a one-time, offline process. Our approach exhibits notable training efficiency, with the generator model converging rapidly even when not leveraging the entire, often vast, training dataset. On an NVIDIA GeForce RTX 4090 GPU, the generator for DeepCorr-300 converged in under 10 minutes, and the model for the larger-scale m-DeepCorr converged within 15 minutes. For our most complex target, DeepCoFFEA, the generator achieved convergence in under 30 minutes, despite training on only a random 1/26th subset of the data per epoch due to the dataset’s scale. These rapid training cycles demonstrate the practical feasibility of deploying Augur and its potential for efficient retraining against emerging attack models.

Real-time Computational Overhead. To validate the feasibility of running Augur as a Tor Pluggable Transport on

client devices, we evaluated its computational footprint on typical consumer hardware (an AMD Ryzen 7 4800H CPU) without GPU acceleration. The tests were conducted entirely on the CPU to simulate a realistic deployment environment. The results are summarized in Table III.

The evaluation confirms Augur’s high efficiency, with 95th percentile generation latencies (13–22 ms) and peak RAM usage (510–554 MB) well within acceptable limits for modern clients. This real-time performance is achieved via our asynchronous, producer-consumer architecture (detailed in Section V-B), wherein the perturbation generator runs in a background worker thread to pre-populate a queue. The main network I/O thread, in turn, simply dequeues these pre-computed instructions, a computationally trivial operation. This architecture ensures the generator’s inference latency does not block the network I/O path or introduce direct packet delay, effectively hiding the computation cost. The high average CPU usage ($\sim 700\%$) is a reflection of this non-blocking background work, demonstrating that Augur effectively leverages available multi-core resources without impeding foreground responsiveness. This asynchronous design, combined with the negligible bandwidth overhead (shown in Table II), establishes Augur’s real-world deployability.

C. Ablation Experiments

1) *Impact of Perturbation Components:* To dissect the individual contributions of Augur’s core components—packet size (padding) and packet timing (delay)—we conducted an ablation study. We trained three generator variants: 1) Time-only, constrained to only generate timing perturbations (δ_{size} is forced to 0); 2) Size-only, constrained to only generate padding (δ_{time} is forced to 0); and 3) Both (Full Augur), the standard model.

We evaluated these variants against all three target attack models, with the results presented in Fig. 7. The ROC curves (Figs. 7a,7c,7e) clearly show that the defensive performance of the Time-only variant is nearly identical to that of the full, dual-component approach. In stark contrast, the Size-only variant’s effectiveness is dramatically diminished, with its ROC curve approaching the undefended baseline. This indicates that timing perturbations alone are sufficient for robust defense.

This finding is corroborated by the quantitative cost analysis in Figs. 7b,7d,7f, which reveal the resource allocation strategy autonomously learned by the full Augur model. Across all adversarial settings, the model preferentially allocates its perturbation budget to the timing dimension, keeping size-related perturbations minimal. For instance, against DeepCoFFEA, the learned L2-norm ratio for timing ($L2_{time}R$) is 0.1398, whereas the ratio for size ($L2_{size}R$) is merely 0.0038—a difference of over 36x.

Collectively, these results provide compelling evidence that timing perturbation is the primary driver of Augur’s defensive efficacy. This pronounced sensitivity is particularly rational for the DeepCoFFEA model, reflecting its inherent architectural reliance on temporal features. DeepCoFFEA’s core “Amplification” mechanism operates by partitioning traffic into time

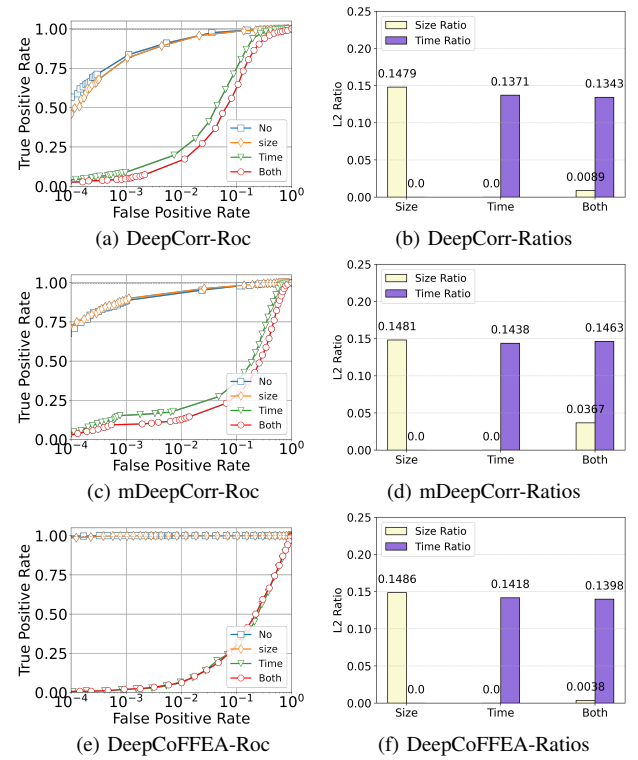


Fig. 7. Ablation study of Augur.

windows based on packet timestamps and then aggregating votes. Consequently, a minor IPD alteration can directly shift a packet’s window assignment, inflicting structural damage that invalidates the voting process. In comparison, altering packet size only localizes the impact within a window, rendering it far less destructive.

Ultimately, the convergence on this efficient “time-heavy, size-light” configuration constitutes an emergent property of end-to-end optimization. The generator autonomously discovers this strategy by balancing defensive efficacy against perturbation costs rather than relying on predefined heuristics. This behavior validates the framework’s adaptive capability to allocate resources dynamically where they exert the maximum disruptive impact.

2) *Impact of Differentiable Window Partitioning:* The discrete windowing mechanism inherent to DeepCoFFEA constitutes a non-differentiable barrier that severs the gradient path from the classification loss to the packet timestamps, fundamentally precluding end-to-end optimization of the temporal perturbation strategy. To quantify the necessity of our proposed differentiable surrogate, we conducted a rigorous ablation study comparing the full Augur model against a baseline generator trained with standard non-differentiable operations. Fig. 8 reveals a decisive divergence in the training dynamics: the differentiable model converges to a lower Cosine Loss compared to the baseline, indicating a superior capability to suppress the correlation features exploited by the attack.

It is critical to observe that the non-differentiable baseline is not entirely ineffective; it achieves a partial reduction in loss. We attribute this residual efficacy to the preservation of gradient flow for the feature values (the specific IPDs and sizes input to the feature extractor), which remain dif-

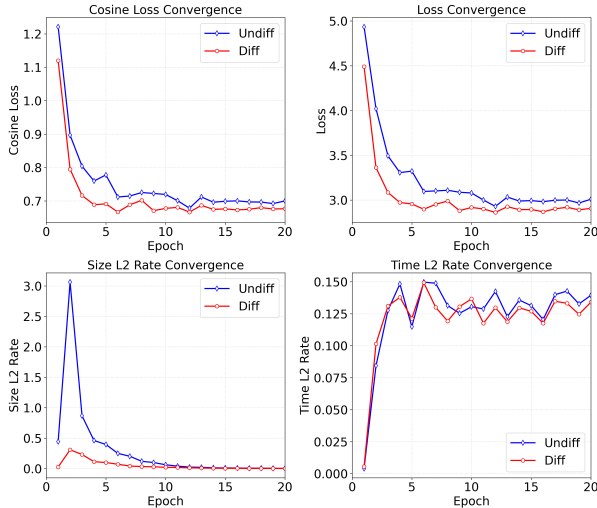


Fig. 8. Impact of the differentiable windowing mechanism on training convergence.

ferentiable even when the window indices are frozen. This implies that the baseline generator learns to perform a limited form of “content-level optimization” by distorting feature values within structurally rigid windows. However, it lacks the structural gradient required to intelligently shift timestamps to alter the window segmentation itself. Our differentiable mechanism restores this critical structural path. The resulting drop in Time L2 Rate confirms that the model transitions from a brute-force distortion of values to a precise, structural manipulation of traffic alignment. This efficiency stems from the mechanism’s intrinsic property of scaling gradients by the inverse of packet intervals, which incentivizes the generator to apply “surgical” shifts in dense traffic regions where minimal perturbations yield maximum boundary disruption. Furthermore, the Size L2 Rate remains virtually unchanged across both experiments. This stability corroborates our hypothesis that the mechanism’s impact is strictly decoupled from the packet size dimension, explicitly targeting the temporal logic without inducing collateral resource consumption. These findings establish that the differentiable windowing mechanism is a fundamental prerequisite for transcending local content optimization and learning globally optimal, structure-aware perturbation strategies.

D. Transferability

A critical measure of a defense’s robustness is its transferability. We evaluate this in a strict black-box setting: a perturbation generator trained *only* on DeepCorr is used to attack DeepCoFFEA, a target with an unseen architecture and dataset. This test is unidirectional and particularly challenging, as the DeepCorr model itself fails when evaluated on DeepCoFFEA’s data—achieving only 0.51 ROC AUC—confirming the data is significantly out-of-distribution.

The results in Fig. 9 demonstrate potent transferability. Our black-box approach, labeled *DeepCorr300* in the figure, successfully degrades DeepCoFFEA’s performance, pushing its ROC curve significantly toward the random guessing baseline. At a stringent 10^{-4} FPR, this attack substantially reduces DeepCoFFEA’s TPR, severely disrupting its performance.

Quantitatively, our black-box approach considerably outperforms standard baselines. As Table IV shows, methods

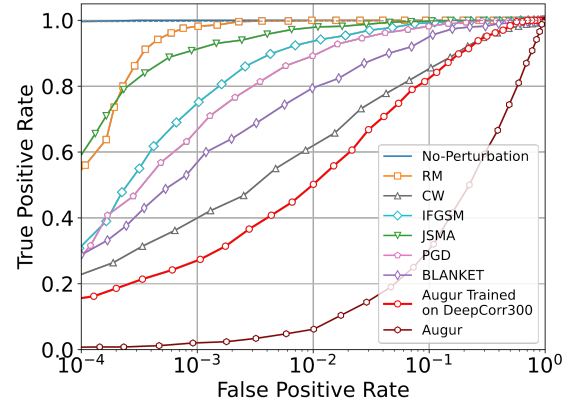


Fig. 9. Perturbations trained on DeepCorr show strong black-box transferability against DeepCoFFEA.

TABLE IV
BLACK-BOX TRANSFERABILITY OF AUGUR AGAINST DEEPCOFFEA

Defense	FPR 10^{-3}			FPR 10^{-4}		
	PRE	REC	F1-Score	PRE	REC	F1-Score
No-Perturbation	0.6658	1.0000	0.7994	0.9524	1.0000	0.9756
BLANKET	0.5318	0.5680	0.5493	0.8494	0.2820	0.4234
C&W	0.4390	0.3960	0.4164	0.8188	0.2260	0.3542
PGD	0.5729	0.6760	0.6202	0.8418	0.2660	0.4043
Black-Box	0.3531	0.2740	0.3086	0.7642	0.1620	0.2673
White-Box	0.0380	0.0200	0.0262	0.1071	0.0060	0.0114

like Random Morphing (RM), C&W, and PGD are far less effective. While the black-box efficacy (F1: 0.2673) does not reach the white-box F1-score of 0.0114, its superior performance over all other baselines suggests the perturbations exploit fundamental, shared vulnerabilities rather than model-specific artifacts.

Critically, this strong defensive outcome was achieved with minimal overhead. The perturbations maintained low average L2 norm ratios of 0.1085 for timing and 0.0097 for size. This demonstrates the generator autonomously learned an efficient “time-heavy, size-light” strategy that is both highly transferable and stealthy.

VII. CONCLUSION

In this paper, we introduce Augur, a new paradigm of predictive defense. By leveraging a Transformer-based forecaster, Augur generates context-aware perturbations in real-time. To enable direct optimization against window-based adversaries, we incorporate a differentiable windowing mechanism. This renders complex window-based attacks to direct end-to-end optimization by bridging the gradient disconnect. Crucially, we realized Augur as a lightweight Tor Pluggable Transport and evaluated it against the SOTA DeepCoFFEA adversary. The defensive performance is compelling : Augur nullifies the attack, slashing its F1-score from 0.9756 to 0.0114 with negligible bandwidth overhead. We have shown that a predictive, context-aware strategy is not an incremental improvement but a necessary evolution for defending modern anonymity systems.

REFERENCES

- [1] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," 2004.
- [2] A. Mani, T. Wilson-Brown, R. Jansen, A. Johnson, and M. Sherr, "Understanding tor usage with privacy-preserving measurement," in *Proceedings of the Internet Measurement Conference 2018*, 2018, pp. 175–187.
- [3] S. J. Murdoch and G. Danezis, "Low-cost traffic analysis of tor," in *2005 IEEE Symposium on Security and Privacy (S&P'05)*. IEEE, 2005, pp. 183–195.
- [4] D. Lopes, J.-D. Dong, P. Medeiros, D. Castro, D. Barradas, B. Portela, J. Vinagre, B. Ferreira, N. Christin, and N. Santos, "Flow correlation attacks on tor onion service sessions with sliding subset sum." in *NDSS*, 2024.
- [5] K. Fredrickson, I. Demertzis, J. P. Hughes, and D. D. Long, "Sparta: Practical anonymity with long-term resistance to traffic analysis," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 1457–1473.
- [6] C. Sendner, J. Stang, A. Dmitrienko, R. Wijewickrama, and M. Jadliwala, "Mirageflow: a new bandwidth inflation attack on tor." *NDSS*, 2024.
- [7] Z. Luo, A. Bhat, K. Nayak, and A. Kate, "Attacking and improving the tor directory protocol," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 3221–3237.
- [8] S. E. Oh, T. Yang, N. Mathews, J. K. Holland, M. S. Rahman, N. Hopper, and M. Wright, "Deepcoffee: Improved flow correlation attacks on tor via metric learning and amplification," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 1915–1932.
- [9] M. Nasr, A. Bahramali, and A. Houmansadr, "Defeating dnn-based traffic analysis systems in real-time with blind adversarial perturbations," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2705–2722.
- [10] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*, 2014, pp. 227–238.
- [11] X. Cai, R. Nithyanand, and R. Johnson, "Cs-bufflo: A congestion sensitive website fingerprinting defense," in *Proceedings of the 13th Workshop on Privacy in the Electronic Society*, 2014, pp. 121–130.
- [12] N. Mathews, J. K. Holland, S. E. Oh, M. S. Rahman, N. Hopper, and M. Wright, "Sok: A critical evaluation of efficient website fingerprinting defenses," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 969–986.
- [13] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2017, pp. 39–57.
- [14] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *arXiv preprint arXiv:1706.06083*, 2017.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [16] J. K. Holland and N. Hopper, "Regulator: A straightforward website fingerprinting defense," *arXiv preprint arXiv:2012.06609*, 2020.
- [17] J.-F. Raymond, "Traffic analysis: Protocols, attacks, design issues, and open problems," in *Designing Privacy Enhancing Technologies: International Workshop on Design Issues in Anonymity and Unobservability Berkeley, CA, USA, July 25–26, 2000 Proceedings*. Springer, 2001, pp. 10–29.
- [18] D. Xue, R. Stanley, P. Kumar, and R. Ensafi, "The discriminative power of cross-layer rtt in fingerprinting proxy traffic." in *NDSS*, 2025.
- [19] T. Tucker, N. Bennett, M. Kotuliak, S. Erni, S. Capkun, K. Butler, and P. Traynor, "Detecting imsi-catchers by characterizing identity exposing messages in cellular traffic," in *Proceedings of the ISOC Networking and Distributed Systems Security (NDSS) Symposium. San Diego, CA, USA*, 2025.
- [20] S. Xi, T. Fu, K. Bu, C. Yang, Z. Chang, W. Chen, Z. Ma, C. Chen, Y. Shen, and K. Ren, "Mineshark: Cryptomining traffic detection at scale." in *NDSS*, 2025.
- [21] K. Man, Z. Wang, Y. Hao, S. Zheng, X. Zhou, Y. Cao, and Z. Qian, "Scad: Towards a universal and automated network side-channel vulnerability detection," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 1861–1876.
- [22] N. Wickramasinghe, A. Shaghaghi, G. Tsudik, and S. Jha, "Sok: Decoding the enigma of encrypted network traffic classifiers," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 1825–1843.
- [23] J. Lee, H. Kim, and M. S. Kang, "Onions got puzzled: On the challenges of mitigating {Denial-of-Service} problems in tor onion services," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 3667–3685.
- [24] Y. Sun, A. Edmundson, L. Vanbever, O. Li, J. Rexford, M. Chiang, and P. Mittal, "Raptor: Routing attacks on privacy in tor," in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 271–286.
- [25] M. Nasr, A. Bahramali, and A. Houmansadr, "Deepcorr: Strong flow correlation attacks on tor using deep learning," in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 1962–1976.
- [26] T. Chawla, S. Mittal, N. Mathews, and M. Wright, "Espresso: Advanced end-to-end flow correlation attacks on tor," in *Proceedings of the 8th Asia-Pacific Workshop on Networking*, 2024, pp. 219–220.
- [27] Y. Yuan, Q. Niu, and Y. Yuan, "Early-mfc: Enhanced flow correlation attacks on tor via multi-view triplet networks with early network traffic," *arXiv preprint arXiv:2503.16847*, 2025.
- [28] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, i still see you: Why efficient traffic analysis countermeasures fail," in *2012 IEEE Symposium on Security and Privacy*. IEEE, 2012, pp. 332–346.
- [29] D. Han, Z. Wang, Y. Zhong, W. Chen, J. Yang, S. Lu, X. Shi, and X. Yin, "Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 8, pp. 2632–2647, 2021.
- [30] M. Shen, K. Ji, Z. Gao, Q. Li, L. Zhu, and K. Xu, "Subverting website fingerprinting defenses with robust traffic representation," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 607–624.
- [31] M. Jiang, B. Cui, J. Fu, T. Wang, L. Yao, and B. K. Bhargava, "Rudolf: An efficient and adaptive defense approach against website fingerprinting attacks based on soft actor-critic algorithm," *IEEE Transactions on Information Forensics and Security*, 2024.
- [32] M. Jiang, B. Cui, J. Fu, T. Wang, and Z. Wang, "Kimerapad: A novel low-overhead real-time defense against website fingerprinting attacks based on deep reinforcement learning," *IEEE Transactions on Network and Service Management*, vol. 21, no. 3, pp. 2944–2961, 2024.
- [33] J. K. Holland, J. Carpenter, S. E. Oh, and N. Hopper, "Detorrent: an adversarial padding-only traffic analysis defense," *arXiv preprint arXiv:2302.02012*, 2023.
- [34] T. Pulls and E. Witwer, "Maybenot: A framework for traffic analysis defenses," in *Proceedings of the 22nd Workshop on Privacy in the Electronic Society*, 2023, pp. 75–89.
- [35] A. Mitseva and A. Panchenko, "Stop, don't click here anymore: boosting website fingerprinting by considering sets of subpages," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 4139–4156.
- [36] Y. Nie, "A time series is worth 64 words: Long-term forecasting with transformers," *arXiv preprint arXiv:2211.14730*, 2022.
- [37] Y. Angel, "obfs4: The obfoursator," The Tor Project, Tech. Rep., October 2014, obfs4 protocol specification. [Online]. Available: <https://gitweb.torproject.org/pluggable-transport/obfs4.git/plain/doc/obfs4-spec.txt>
- [38] I. Homoliak, M. Teknos, M. Ochoa, D. Breitenbacher, S. Hosseini, and P. Hanacek, "Improving network intrusion detection classifiers by non-payload-based exploit-independent obfuscations: An adversarial approach," *arXiv preprint arXiv:1805.02684*, 2018.
- [39] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," in *Artificial intelligence safety and security*. Chapman and Hall/CRC, 2018, pp. 99–112.
- [40] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, "The limitations of deep learning in adversarial settings," in *2016 IEEE European symposium on security and privacy (EuroS&P)*. IEEE, 2016, pp. 372–387.
- [41] R. Sheatsley, N. Papernot, M. J. Weisman, G. Verma, and P. McDaniel, "Adversarial examples for network intrusion detection systems," *Journal of Computer Security*, vol. 30, no. 5, pp. 727–752, 2022.